

Are You Getting What You Pay For?

Auditing Model Substitution in LLM APIs

Why Software Tests Fail and Hardware Attestation Wins

Based on: arxiv.org/abs/2504.04715

Commercial LLM API providers may covertly substitute cheaper models.
Software-based detection is unreliable. TEEs offer a provably secure alternative.

The Model Substitution Problem

Economic Incentives Drive Covert Swaps

The Black-Box Trust Gap

Providers charge for access to specific models (e.g., Llama-3-70B, Qwen2-72B), but the API's black-box nature offers no guarantee the advertised model is actually served.

Formal framing: Given API samples, can an auditor distinguish $P_{\text{spec}}(y|x)$ from $P_{\text{actual}}(y|x)$?

① Smaller Model

e.g., 8B served instead of 70B

② Quantized Variant

FP8 / INT8 instead of FP16

③ Fine-tuned / Updated

Silently updated weights

④ Different Family

Entirely different architecture

Hypothesis Testing Framework

H_0 : API serves the specified model — H_1 : API serves a substitute

The auditor must distinguish these distributions using only query access — a fundamentally constrained setting.

FP8 Quantization Preserves Most Accuracy

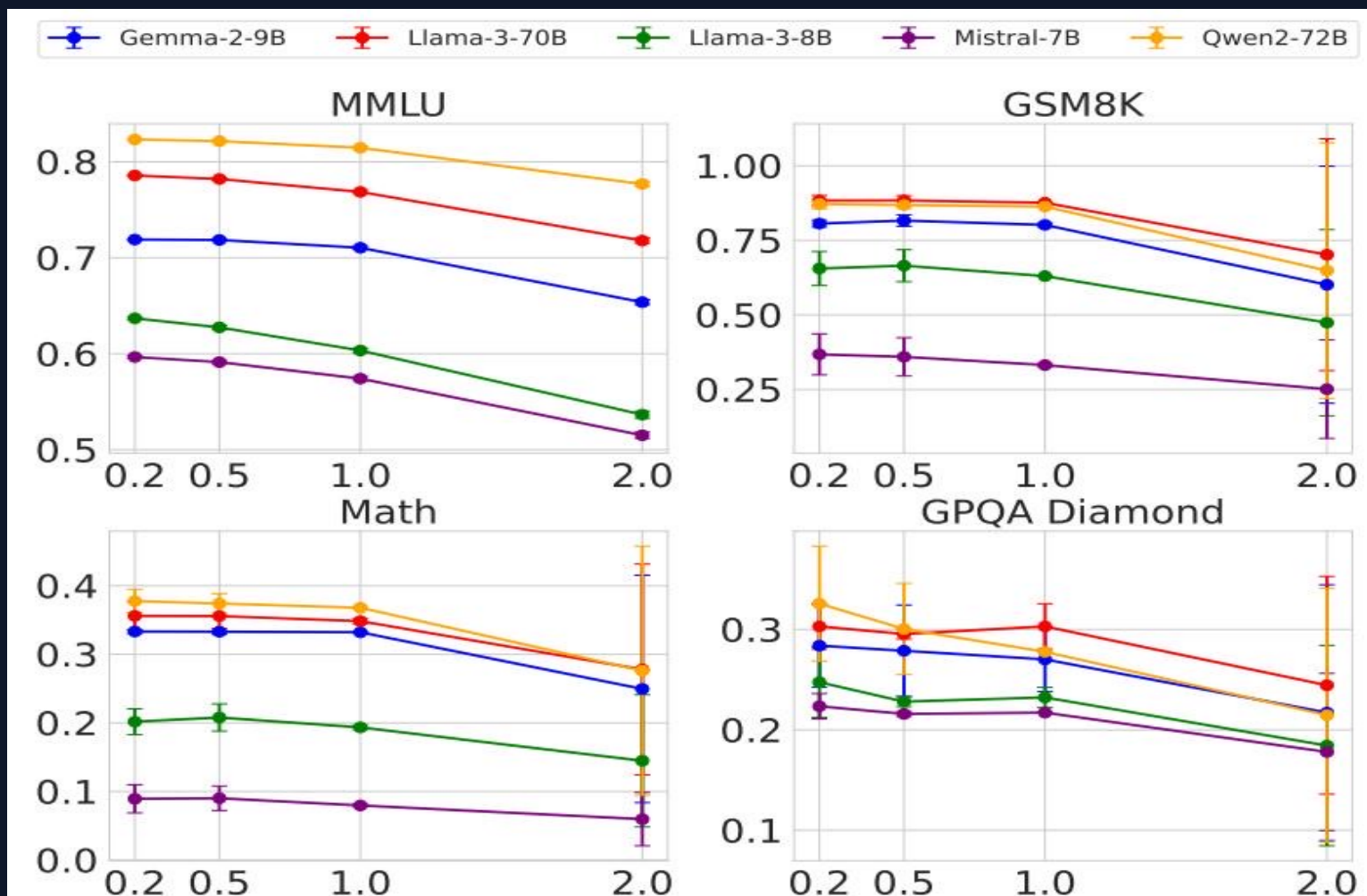
Performance drops are small — often within error bars — making substitution hard to detect

Model	MMLU	GSM8K	MATH	GPQA Diamond
Llama-3-8B-Instruct	62.69 ± 0.18	61.14 ± 3.47	20.65 ± 3.43	22.62 ± 0.22
Llama-3-8B-Instruct-FP8	62.43 ± 0.26	60.90 ± 4.10	14.91 ± 2.49	20.14 ± 0.24
Llama-3-70B-Instruct	78.05 ± 0.08	88.06 ± 1.44	35.69 ± 1.33	29.60 ± 0.51
Llama-3-70B-Instruct-FP8	77.88 ± 0.13	87.35 ± 1.24	35.75 ± 1.16	33.12 ± 0.30
Gemma-2-9b-it	71.86 ± 0.08	81.80 ± 1.35	33.41 ± 0.28	28.64 ± 2.97
Gemma-2-9b-it-FP8	71.92 ± 0.11	79.41 ± 1.14	32.53 ± 0.34	27.81 ± 3.22
Qwen2-72B-Instruct	82.18 ± 0.08	86.72 ± 1.00	37.39 ± 1.41	29.93 ± 2.71
Qwen2-72B-Instruct-FP8	81.98 ± 0.08	86.82 ± 0.97	37.67 ± 1.39	31.08 ± 1.96
Mistral-7B-Instruct-v0.3	59.15 ± 0.10	35.90 ± 4.54	8.94 ± 1.24	21.60 ± 0.17
Mistral-7B-Instruct-v0.3-FP8	58.77 ± 0.13	32.20 ± 4.02	7.68 ± 1.23	22.72 ± 0.19

⚠ For most models, FP8 scores fall within error bars of the original — especially on MMLU and GSM8K.

Temperature Scaling Degrades All Models

Higher temperature amplifies performance gaps but increases noise



Key Observation

At temperature 2.0, all models' performance degrades sharply. The gap between original and FP8 narrows or becomes noisy, complicating any statistical detection strategy.

Temperatures Tested

T = 0.2 — near-deterministic
T = 0.5 — moderate sampling
T = 1.0 — standard
T = 2.0 — high variance

Implication

Temperature is a confounder — providers can tune it to further obscure substitution signals.

Log-Probability Fingerprinting Fails

Inference nondeterminism defeats metadata-based detection

Model	BERT Acc	T5 Acc	GPT2 Acc	LLM2Vec Acc
Llama3-70B-Instruct-FP8	50.55	50.10	50.45	49.90
Llama3-70B-Instruct-INT8	51.60	49.90	51.30	50.25
Gemma2-9b-it-FP8	49.95	50.30	51.20	49.80
Gemma2-9b-it-INT8	49.00	49.70	51.65	49.55
Mistral-7b-v3-Instruct-FP8	50.55	49.75	48.70	48.75
Mistral-7b-v3-Instruct-INT8	49.70	50.75	50.50	51.15
Qwen2-72B-Instruct-FP8	50.05	50.25	48.75	49.80
Qwen2-72B-Instruct-INT8	50.75	50.55	49.45	50.20

The Same Model ≠ The Same Log Probs

Llama-3-8B produces different token-level log probabilities when served via different frameworks and hardware:

- transformers 4.40 vs 4.50
- vLLM 0.6.2 vs 0.8.2
- A100 vs H100 GPUs

Why This Matters

An auditor comparing local reference log probs against the API cannot distinguish infrastructure variation from actual model substitution.

Conclusion

Log-probability fingerprinting is unreliable as a detection method in production inference environments.

Randomized Routing Makes Detection Query-Prohibitive

Adversarial countermeasures further undermine software-only auditing

① Randomized Substitution

The provider routes a fraction p of queries to the cheaper model, serving the genuine model for the remaining $(1-p)$ queries.

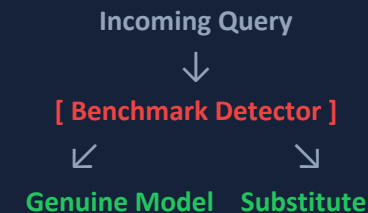
$$P_{\text{mixed}} = (1-p) \cdot P_{\text{spec}} + p \cdot P_{\text{sub}}$$

As $p \rightarrow 0$, the mixed distribution converges to the specified model's distribution.

Detection requires exponentially more queries as p decreases — making auditing economically infeasible at low substitution rates.

② Benchmark Evasion

Providers can detect known benchmark prompts (MMLU, GSM8K, etc.) and route them to the genuine model while serving substitutes for regular user traffic.



This makes benchmark-based verification fundamentally unreliable.

Combined effect: adversarial strategies make software-only detection fundamentally query-prohibitive.

Software-Only Auditing Methods Are Fundamentally Unreliable

Three independent failure modes make detection unreliable in practice

Method Type	Core Weakness	Practical Limitation
Text-Based Statistical Tests (e.g., MMD, distributional divergence)	Require impractically large query budgets for reliable detection	Fail against subtle substitutions (e.g., FP8) where output distributions are nearly identical
Log-Probability Fingerprinting (token-level metadata comparison)	Defeated by production-level inference nondeterminism	Software versions (transformers, vLLM) and GPU hardware (A100 vs H100) cause irreducible variation
Adversarial Countermeasures (benchmark evasion, randomized routing)	Provider actively evades detection rather than passively being tested	Benchmark prompts can be detected and routed to the genuine model; low-rate mixing is undetectable

Verdict: No software-only method provides reliable detection. Each fails independently, and adversarial countermeasures compound the problem. A fundamentally different approach is needed.

Trusted Execution Environments: Provable Model Integrity

Hardware-level cryptographic attestation guarantees the correct model is served

What TEEs Provide

- ✓ **Cryptographic Attestation** — Prove specific model weights are loaded
- ✓ **Code Integrity** — Verify the correct inference code is running
- ✓ **Tamper Resistance** — Hardware-secured enclaves prevent modification
- ✓ **Modest Overhead** — Practical performance cost for production use

TEEs vs Software Methods

- Provable:** Cryptographic guarantee vs statistical estimation
- Efficient:** No massive query budget required
- Robust:** Immune to adversarial routing and evasion
- Deployable:** NVIDIA confidential computing on H100 GPUs

Why Not Zero-Knowledge Proofs?

ZKPs can theoretically verify computation correctness, but are computationally prohibitive for large language models. TEEs achieve the same integrity guarantee with practical, deployable hardware that exists today.

Deployment Path: NVIDIA Confidential Computing on H100 GPUs enables TEE-based attestation for LLM inference — available today for production workloads.

Key Takeaways

Close the Verification Gap with Hardware Attestation

①

Model substitution is economically motivated and practically undetectable via software

Providers face strong incentives to serve cheaper alternatives; the API's black-box nature hides the swap.

②

FP8 quantization barely affects benchmarks

Performance drops fall within error bars for most models and tasks, making quantized substitution nearly invisible.

③

Inference nondeterminism kills log-probability fingerprinting

Software versions and GPU hardware cause irreducible variation that masks any substitution signal.

④

Randomized substitution defeats statistical tests

Low-rate mixing converges to the genuine distribution, requiring exponentially more queries for detection.

⑤

TEEs are the only currently viable path to verifiable LLM APIs

Hardware-secured cryptographic attestation provides provable, efficient, and robust integrity guarantees — deployable today.

Source: arxiv.org/abs/2504.04715 — Demand TEE-backed attestation from your LLM API providers.